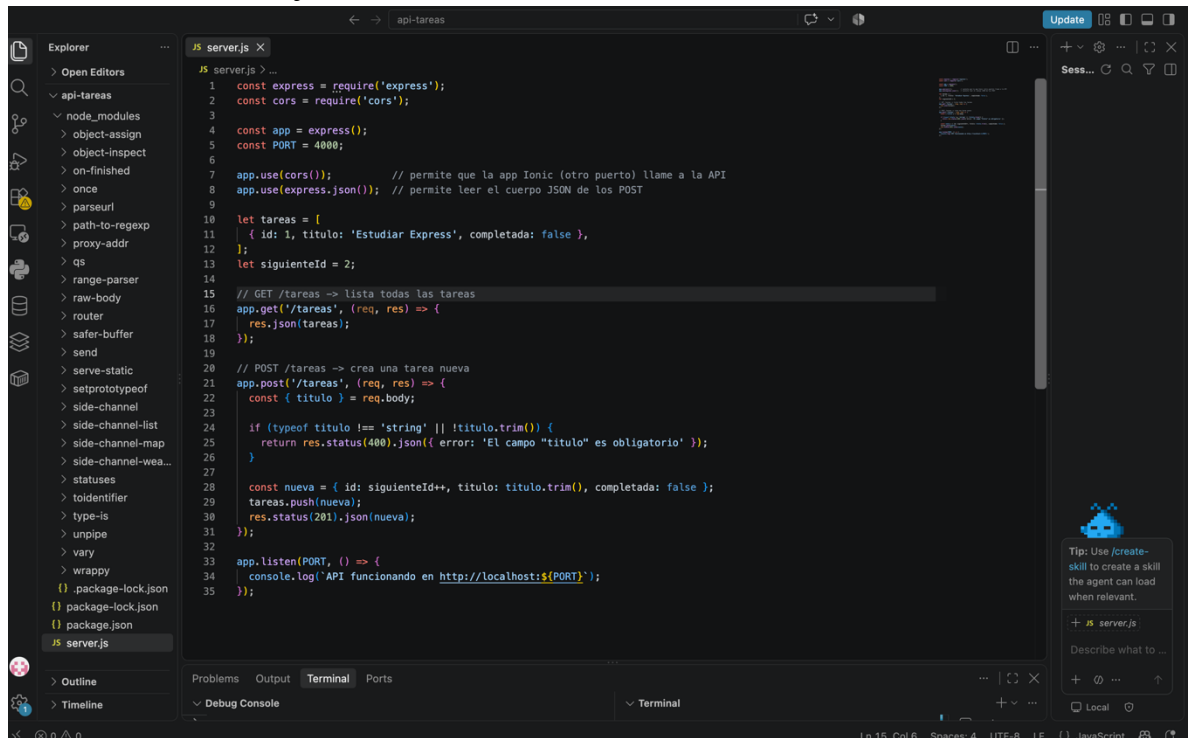


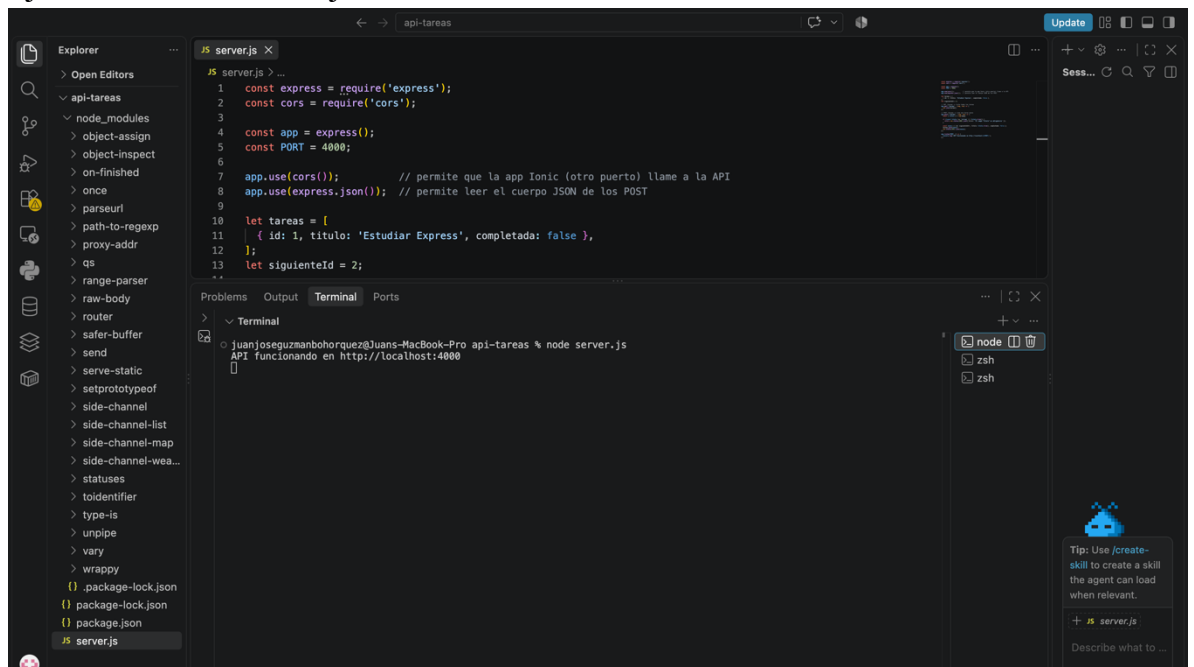
1. Crea el archivo server.js



The screenshot shows the Visual Studio Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'api-tareas' with a 'node_modules' directory. The code editor displays the content of 'server.js', which is a Node.js application using Express.js. The code includes imports for 'express' and 'cors', initialization of an Express app, and configuration of CORS and JSON body parsing. It defines a 'tareas' array with one task and a 'siguienteId' variable. The app has two endpoints: a GET endpoint for listing tasks and a POST endpoint for creating new tasks. The server is configured to listen on port 4000.

```
1 const express = require('express');
2 const cors = require('cors');
3
4 const app = express();
5 const PORT = 4000;
6
7 app.use(cors()); // permite que la app Ionic (otro puerto) llame a la API
8 app.use(express.json()); // permite leer el cuerpo JSON de los POST
9
10 let tareas = [
11   { id: 1, titulo: 'Estudiar Express', completada: false },
12 ];
13 let siguienteId = 2;
14
15 // GET /tareas -> lista todas las tareas
16 app.get('/tareas', (req, res) => {
17   res.json(tareas);
18 });
19
20 // POST /tareas -> crea una tarea nueva
21 app.post('/tareas', (req, res) => {
22   const { titulo } = req.body;
23
24   if (typeof titulo !== 'string' || !titulo.trim()) {
25     return res.status(400).json({ error: 'El campo "titulo" es obligatorio' });
26   }
27
28   const nueva = { id: siguienteId++, titulo: titulo.trim(), completada: false };
29   tareas.push(nueva);
30   res.status(201).json(nueva);
31 });
32
33 app.listen(PORT, () => {
34   console.log('API funcionando en http://localhost:${PORT}');
35 });
```

Ejecutar con node server.js



The screenshot shows the Visual Studio Code editor with the same 'server.js' file open. The 'Terminal' panel at the bottom is active, showing the command 'node server.js' being executed. The output of the command is 'API funcionando en http://localhost:4000'. The file explorer on the left shows the 'server.js' file selected.

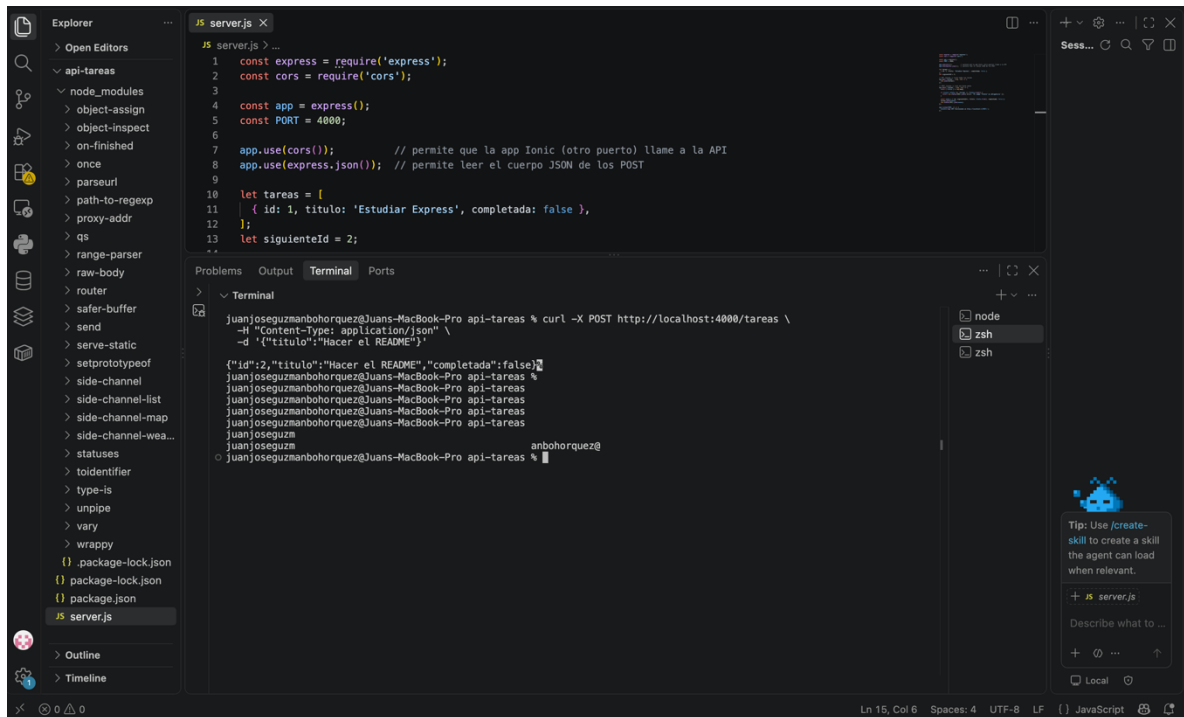
```
node server.js
API funcionando en http://localhost:4000
```

2. Probar los endpoints

Con `curl -X POST http://localhost:4000/tareas \`

`-H "Content-Type: application/json" \`

-d '{"titulo":"Hacer el README"}'

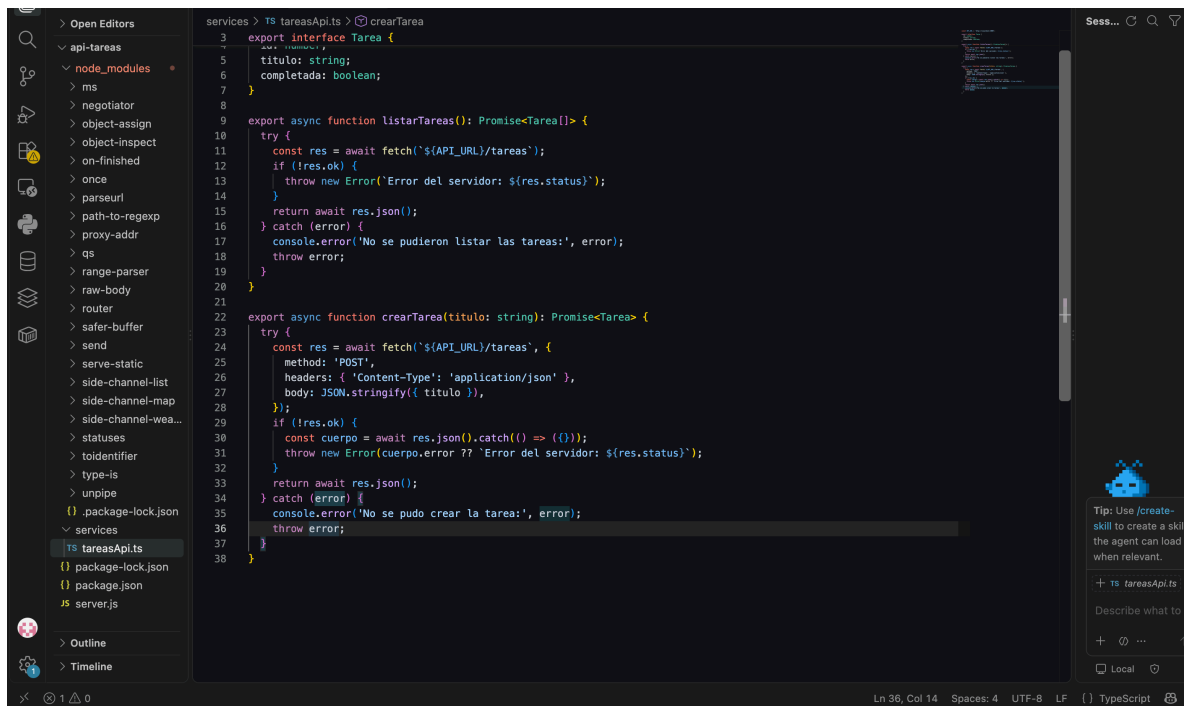


The screenshot shows a VS Code editor with a file named `server.js` open. The file contains an Express.js server setup. A REST client request is defined in the file, and the terminal shows the command being executed: `curl -X POST http://localhost:4000/tareas \-d '{"titulo":"Hacer el README"}'`. The terminal output shows the response from the server: `{id:2,titulo:"Hacer el README",completada:false}`. The Explorer sidebar on the left shows the file structure, and the Terminal sidebar on the right shows the command being executed.

```
1 const express = require('express');
2 const cors = require('cors');
3
4 const app = express();
5 const PORT = 4000;
6
7 app.use(cors()); // permite que la app Ionic (otro puerto) llame a la API
8 app.use(express.json()); // permite leer el cuerpo JSON de los POST
9
10 let tareas = [
11   { id: 1, titulo: 'Estudiar Express', completada: false },
12 ];
13 let siguienteId = 2;
```

```
juanjoseguzmanbohorquez@Juans-MacBook-Pro api-tareas % curl -X POST http://localhost:4000/tareas \
-d '{"titulo":"Hacer el README"}'
{"id:2,titulo:"Hacer el README",completada:false}
juanjoseguzmanbohorquez@Juans-MacBook-Pro api-tareas %
```

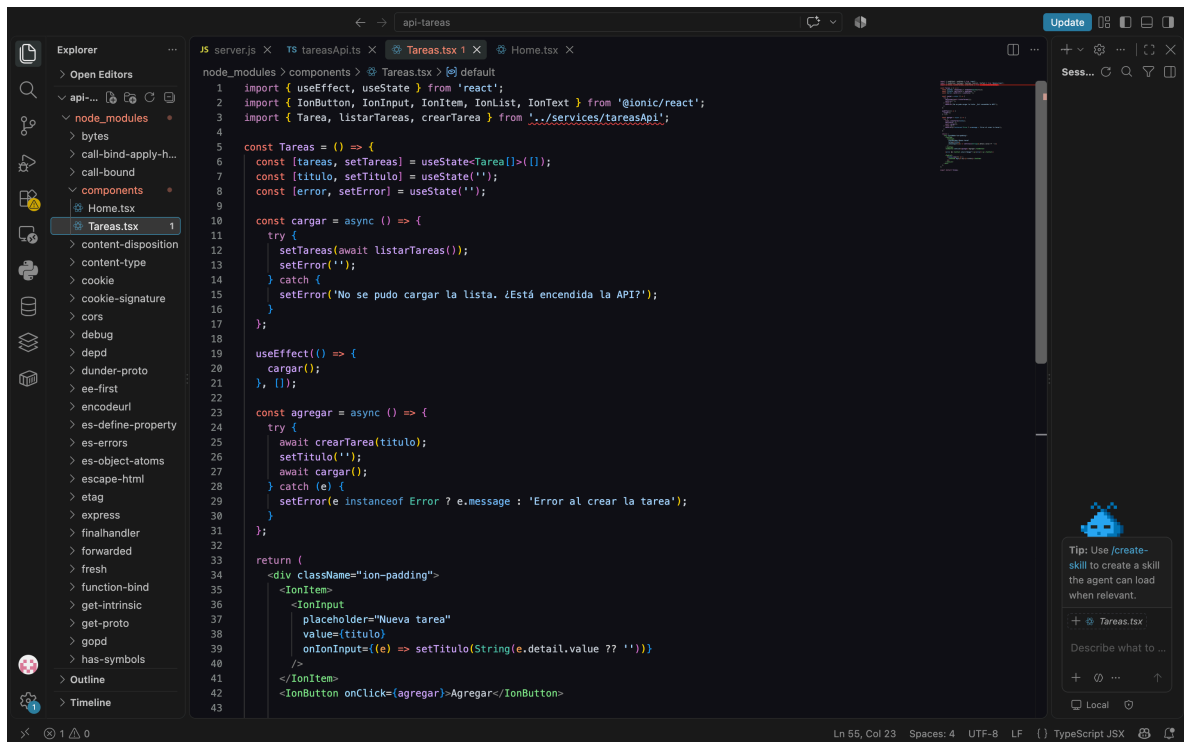
3. Funciones fetch con manejo de errores



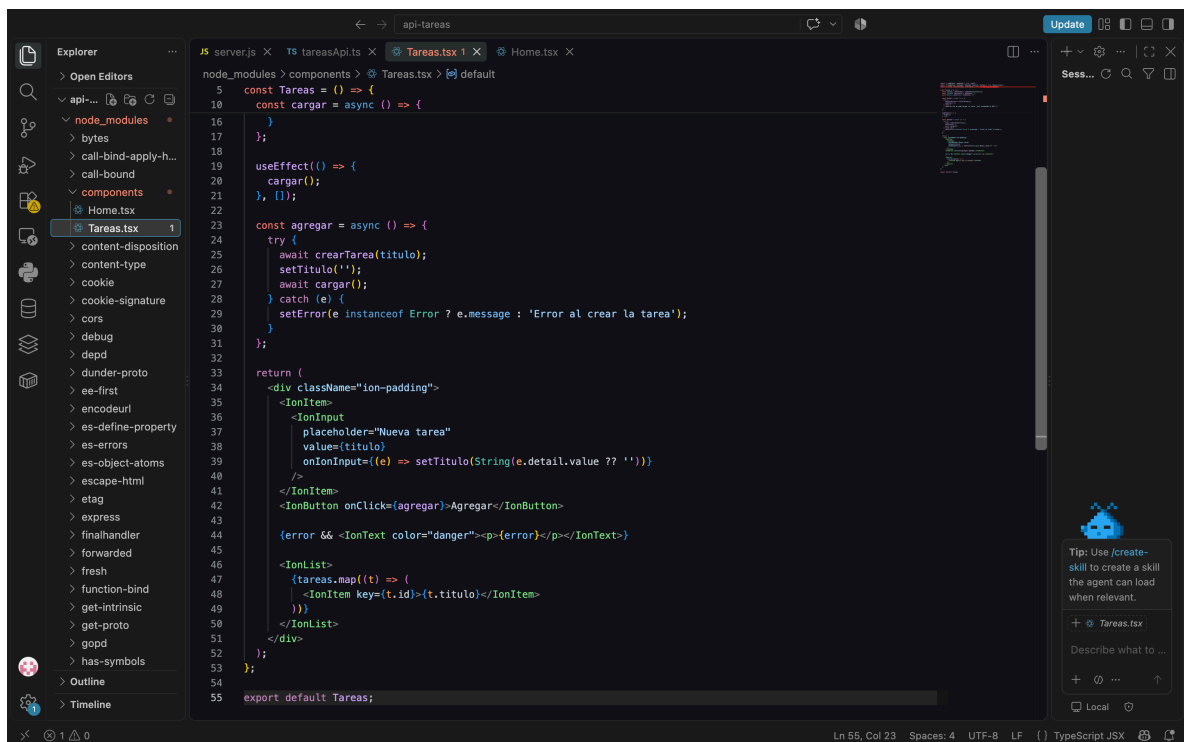
The screenshot shows a VS Code editor with a file named `ts tareasApi.ts` open. The file contains TypeScript code for a REST client. The code defines an interface `Tarea` and two async functions: `listarTareas` and `crearTarea`. Both functions use `fetch` to interact with the API and include error handling. The Explorer sidebar on the left shows the file structure, and the Terminal sidebar on the right shows the command being executed: `curl -X POST http://localhost:4000/tareas \-d '{"titulo":"Hacer el README"}'`.

```
3 export interface Tarea {
4   titulo: string;
5   completada: boolean;
6 }
7
8
9 export async function listarTareas(): Promise<Tarea[]> {
10   try {
11     const res = await fetch(`${API_URL}/tareas`);
12     if (!res.ok) {
13       throw new Error('Error del servidor: ${res.status}');
14     }
15     return await res.json();
16   } catch (error) {
17     console.error('No se pudieron listar las tareas:', error);
18     throw error;
19   }
20 }
21
22 export async function crearTarea(titulo: string): Promise<Tarea> {
23   try {
24     const res = await fetch(`${API_URL}/tareas`, {
25       method: 'POST',
26       headers: { 'Content-Type': 'application/json' },
27       body: JSON.stringify({ titulo }),
28     });
29     if (!res.ok) {
30       const cuerpo = await res.json().catch(() => ({}));
31       throw new Error(cuerpo.error ?? 'Error del servidor: ${res.status}');
32     }
33     return await res.json();
34   } catch (error) {
35     console.error('No se pudo crear la tarea:', error);
36     throw error;
37   }
38 }
```

4. Usarlas desde la app



```
1 import { useEffect, useState } from 'react';
2 import { IonButton, IonInput, IonItem, IonList, IonText } from '@ionic/react';
3 import { Tarea, listarTareas, crearTarea } from '../services/tareasApi';
4
5 const Tareas = () => {
6   const [tareas, setTareas] = useState<Tarea[]>([]);
7   const [titulo, setTitulo] = useState('');
8   const [error, setError] = useState('');
9
10  const cargar = async () => {
11    try {
12      setTareas(await listarTareas());
13      setError('');
14    } catch {
15      setError('No se pudo cargar la lista. ¿Está encendida la API?');
16    }
17  };
18
19  useEffect(() => {
20    cargar();
21  }, []);
22
23  const agregar = async () => {
24    try {
25      await crearTarea(titulo);
26      setTitulo('');
27      await cargar();
28    } catch (e) {
29      setError(e instanceof Error ? e.message : 'Error al crear la tarea');
30    }
31  };
32
33  return (
34    <div className="ion-padding">
35      <IonItem>
36        <IonInput
37          placeholder="Nueva tarea"
38          value={titulo}
39          onIonInput={e => setTitulo(String(e.detail.value ?? ''))}
40        />
41      </IonItem>
42      <IonButton onClick={agregar}>Agregar</IonButton>
43    </div>
44  );
45
46  <div>
47    <IonList>
48      {tareas.map(t => (
49        <IonItem key={t.id}>{t.titulo}</IonItem>
50      ))}
51    </IonList>
52  </div>
53 </div>
54 </div>
55 export default Tareas;
```



```
5 const Tareas = () => {
10  const cargar = async () => {
16  }
17  };
18
19  useEffect(() => {
20    cargar();
21  }, []);
22
23  const agregar = async () => {
24    try {
25      await crearTarea(titulo);
26      setTitulo('');
27      await cargar();
28    } catch (e) {
29      setError(e instanceof Error ? e.message : 'Error al crear la tarea');
30    }
31  };
32
33  return (
34    <div className="ion-padding">
35      <IonItem>
36        <IonInput
37          placeholder="Nueva tarea"
38          value={titulo}
39          onIonInput={e => setTitulo(String(e.detail.value ?? ''))}
40        />
41      </IonItem>
42      <IonButton onClick={agregar}>Agregar</IonButton>
43
44      {error && <IonText color="danger"><p>{error}</p></IonText>}
45
46      <IonList>
47        {tareas.map(t => (
48          <IonItem key={t.id}>{t.titulo}</IonItem>
49        ))}
50      </IonList>
51    </div>
52  );
53 </div>
54 </div>
55 export default Tareas;
```

Agrégallo en Home.tsx



```
1 import Tareas from '../components/Tareas';
2
3 // dentro de <IonContent>;
4 <Tareas />
```